



AFD & AFN

Taller 1

Tomas Parra

Teoría de lenguajes y laboratorio

Profesora: Luz Páez

Medellín, 2026

Facultad de Ingeniería

Universidad de Antioquia

AFD

A) Registro de asistencia

1. Alfabeto

$$\Sigma = \{P, A\}$$

2. Estados:

- q_0 : Inicial, aun no se ha registrado ninguna P
- q_1 : Aceptación, representa que ya se registro una persona

3. Tabla de transición

Estado	P	A
$\rightarrow q_0$	q_1	q_0
$* q_1$	q_1	q_1

4. Pruebas

Cadena	Resultado esperado
P	Valido
AP	Valido
AAP	Valido
PAAP	Valido
AAA	No valido

5. Código

```
6. /*
7.  * AFD - Ejercicio 1: Registro de asistencia
8.  * Alfabeto: {P, A}
9.  * Acepta las cadenas que tengan por lo menos una P.
10. */
11.
12. #include <iostream>
13. #include <string>
14. using namespace std;
15.
16. int main() {
17.     int cantidad;
18.
19.     cout << "AFD - Registro de asistencia (al menos una P)\n";
20.     cout << "Ingrese el numero de cadenas a probar: ";
21.     cin >> cantidad;
22. }
```

```

23.     for (int i = 0; i < cantidad; i++) {
24.         string cadena;
25.         cout << "Cadena " << i + 1 << ": ";
26.         cin >> cadena;
27.
28.         int estado = 0;
29.         bool valida = true;
30.
31.         for (int j = 0; j < cadena.length(); j++) {
32.             char letra = cadena[j];
33.
34.             if (letra != 'P' && letra != 'A') {
35.                 valida = false;
36.                 break;
37.             }
38.
39.             if (estado == 0) {
40.                 if (letra == 'P') {
41.                     estado = 1;
42.                 }
43.             }
44.         }
45.
46.         if (!valida) {
47.             cout << " -> Cadena con simbolos fuera del alfabeto
{P,A}. Rechazada.\n";
48.         }
49.         else if (estado == 1) {
50.             cout << " -> ACEPTADA (el estudiante estuvo presente al
menos una vez)\n";
51.         }
52.         else {
53.             cout << " -> RECHAZADA\n";
54.         }
55.     }
56.
57.     return 0;
58. }
59.

```

B) Código de acceso

1. Alfabeto

$$\Sigma = \{0,1\}$$

2. Estados:

A: Inicial, el último símbolo leído no aporta al patrón "01" (fue 1, o aún no se ha leído nada).

B: el último símbolo leído fue 0 (posible inicio del patrón).

C: Aceptación, los dos últimos símbolos leídos fueron exactamente "01" (patrón completo).

3. Tabla de transición

Estado	0	1
→ A	B	A
B	B	C
* A	B	A

4. Pruebas

Cadena	Resultado esperado
01	Valido
101	Valido
1001	Valido
1101	Valido
010	No valido
111	No valido

5. Código

```

6. 3
7. /*
8.  * AFD - Ejercicio 2: Codigo de acceso
9.  * Alfabeto: {0, 1}
10. * Acepta las cadenas que terminan en 01.
11. */
12.
13. #include <iostream>
14. #include <string>
15. using namespace std;
16.
17. int main() {
18.     int cantidad;
19.
20.     cout << "AFD - Codigo de acceso (debe terminar en 01)\n";
21.     cout << "Ingrese el numero de cadenas a probar: ";
22.     cin >> cantidad;
23.

```

```

24.     for (int i = 0; i < cantidad; i++) {
25.         string cadena;
26.         cout << "Cadena " << i + 1 << ": ";
27.         cin >> cadena;
28.
29.         int estado = 0;
30.         bool valida = true;
31.
32.         for (int j = 0; j < cadena.length(); j++) {
33.             char letra = cadena[j];
34.
35.             if (letra != '0' && letra != '1') {
36.                 valida = false;
37.                 break;
38.             }
39.
40.             if (estado == 0) {
41.                 if (letra == '0')
42.                     estado = 1;
43.                 else
44.                     estado = 0;
45.             }
46.             else if (estado == 1) {
47.                 if (letra == '0')
48.                     estado = 1;
49.                 else
50.                     estado = 2;
51.             }
52.             else {
53.                 if (letra == '0')
54.                     estado = 1;
55.                 else
56.                     estado = 0;
57.             }
58.         }
59.
60.         if (!valida) {
61.             cout << "    -> Cadena con simbolos fuera del alfabeto
{0,1}. Rechazada.\n";
62.         }
63.         else if (estado == 2) {
64.             cout << "    -> ACEPTADA (el codigo termina en 01)\n";
65.         }
66.         else {
67.             cout << "    -> RECHAZADA\n";

```

```

68.     }
69. }
70.
71.     return 0;
72.}
73.

```

C) Registro de un evento

1. Alfabeto

$\Sigma = \{R, C, P\}$

2. Estados:

- q0: Inicial, esperando la primera R.
- q1: Ya se registró R; esperando C.
- q2: Ya se registró R,C; esperando P.
- q3: Aceptación, se completó correctamente $R \rightarrow C \rightarrow P$ (estado de aceptación); admite nuevas R o C.
- qE: Estado de error/trampa; la secuencia se rompió y la cadena queda rechazada.

3. Tabla de transición

Estado	R	C	P
$\rightarrow q0$	q1	qE	qE
q1	qE	q2	qE
q2	qE	qE	q3
* q3	q3	q3	qE
qE	qE	qE	qE

4. Pruebas

Cadena	Resultado esperado
RCP	Valido
RCCP	No valido
RCPP	No valido
RCRCP	No valido
RCPR	Valido
CRP	No valido

RP	No valido
----	-----------

Explicación de cada caso:

- RCP: sigue exactamente el orden $R \rightarrow C \rightarrow P$; termina en q_3 (aceptación). Válido.
- RCCP: tras R,C llega a q_2 esperando P, pero llega otra C; pasa a q_E . Ya no puede aceptar. No válido.
- RCPP: completa RCP y llega a q_3 , pero la P siguiente inmediatamente después de la aceptación lleva a q_E . No válido.
- RCRCP: tras R,C llega a q_2 esperando P, pero llega una R; pasa a q_E y ya no se recupera aunque después venga C,P. No válido.
- RCPR: completa RCP (llega a q_3) y luego una R mantiene el automata en q_3 . Válido.
- CRP: el primer símbolo debe ser R; al llegar C desde q_0 pasa a q_E . No válido.
- RP: tras R llega a q_1 esperando C, pero llega P; pasa a q_E . No válido.

5. Código

```

6.  /*
7.   * AFD - Ejercicio 3: Registro de un evento
8.   * La cadena debe tener la secuencia R, C, P en ese orden.
9.   */
10.
11. #include <iostream>
12. #include <string>
13. using namespace std;
14.
15. int main() {
16.     int cantidad;
17.
18.     cout << "AFD - Registro de evento (secuencia R -> C -> P)\n";
19.     cout << "Ingrese el numero de cadenas a probar: ";
20.     cin >> cantidad;
21.
22.     for (int i = 0; i < cantidad; i++) {
23.         string cadena;
24.         cout << "Cadena " << i + 1 << ": ";
25.         cin >> cadena;
26.
27.         int estado = 0;
28.         bool valida = true;
29.
30.         for (int j = 0; j < cadena.length(); j++) {
31.             char letra = cadena[j];
32.

```

```

33.         if (letra != 'R' && letra != 'C' && letra != 'P') {
34.             valida = false;
35.             break;
36.         }
37.
38.         if (estado == 0) {
39.             if (letra == 'R')
40.                 estado = 1;
41.             else
42.                 estado = 4;
43.         }
44.         else if (estado == 1) {
45.             if (letra == 'C')
46.                 estado = 2;
47.             else
48.                 estado = 4;
49.         }
50.         else if (estado == 2) {
51.             if (letra == 'P')
52.                 estado = 3;
53.             else
54.                 estado = 4;
55.         }
56.         else if (estado == 3) {
57.             if (letra == 'R' || letra == 'C')
58.                 estado = 3;
59.             else
60.                 estado = 4;
61.         }
62.         else {
63.             estado = 4;
64.         }
65.     }
66.
67.     if (!valida) {
68.         cout << "    -> Cadena con simbolos fuera del alfabeto
{R,C,P}. Rechazada.\n";
69.     }
70.     else if (estado == 3) {
71.         cout << "    -> ACEPTADA (se registro, confirmo y participo
en orden)\n";
72.     }
73.     else {
74.         cout << "    -> RECHAZADA\n";
75.     }

```



```

76.     }
77.
78.     return 0;
79. }
80.

```

AFN

A) Registro para un evento

1. Alfabeto

$\Sigma = \{A, B, C\}$

2. Estados:

- q_0 : Inicial, aun no se ha leído la primer letra .
- q_1 : Aceptación, se leyó la A inicial, a partir de aca se acepta cualquier combinación.

3. Tabla de transición

Estado	R	C	P
$\rightarrow q_0$	$\{q_1\}$	$\emptyset$	$\emptyset$
$* q_1$	$\{q_1\}$	$\{q_1\}$	$\{q_1\}$

Al recibir A en q_0 , el automata avanza a q_1 (aceptación). Si en q_0 se recibe B o C, no hay transición definida ($\emptyset$) y la cadena se rechaza de inmediato, pues no comenzó con A. Una vez en q_1 , cualquiera de los tres símbolos mantiene el automata en q_1 .

4. Pruebas

Cadena	Resultado esperado
A	Valido
AB	Valido
ACBA	Valido
AABC	Valido
B	No valido
C	No valido
BA	No valido
ACB	No valido

5. Código

```
6.  /*
7.   * AFN - Ejercicio 1: Codigo de inscripcion
8.   * La cadena debe comenzar con A.
9.   */
10.
11. #include <iostream>
12. #include <string>
13. using namespace std;
14.
15. bool simularAFN(string cadena) {
16.     int estado = 0;
17.
18.     for (int i = 0; i < cadena.length(); i++) {
19.         char letra = cadena[i];
20.
21.         if (estado == 0) {
22.             if (letra == 'A')
23.                 estado = 1;
24.             else
25.                 return false;
26.         }
27.         else {
28.             if (letra != 'A' && letra != 'B' && letra != 'C')
29.                 return false;
30.         }
31.     }
32.
33.     if (estado == 1)
34.         return true;
35.
36.     return false;
37. }
38.
39. int main() {
40.     int cantidad;
41.
42.     cout << "AFN - Codigo de inscripcion (debe comenzar con A)\n";
43.     cout << "Ingrese el numero de cadenas a probar: ";
44.     cin >> cantidad;
45.
46.     for (int i = 0; i < cantidad; i++) {
47.         string cadena;
```

```

48.         cout << "Cadena " << i + 1 << ": ";
49.         cin >> cadena;
50.
51.         bool valida = !cadena.empty();
52.
53.         for (int j = 0; j < cadena.length(); j++) {
54.             if (cadena[j] != 'A' && cadena[j] != 'B' && cadena[j] !=
55.                 'C') {
56.                 valida = false;
57.                 break;
58.             }
59.
60.             if (!valida) {
61.                 cout << " -> Cadena vacia o con simbolos fuera de
62.                     {A,B,C}. Rechazada.\n";
63.             }
64.             else if (simularAFN(cadena)) {
65.                 cout << " -> ACEPTADA (comienza con A)\n";
66.             }
67.             else {
68.                 cout << " -> RECHAZADA\n";
69.             }
70.         }
71.         return 0;
72.     }
73.

```

B) Categoría de un producto

1. Alfabeto

$\Sigma = \{A, B, P\}$

2. Estados:

- q0: Inicial, aun no se ha visto ninguna P
- q1: Aceptación, ya se vio al menos una P.

3. Tabla de transición

Estado	A	B	P
$\rightarrow q_0$	{q0}	{q0}	{q0,q1}
* q1	{q1}	{q1}	{q1}

A y B nunca sacan a la cadena de q_0 (símbolos irrelevantes para la condición) ni la sacan de q_1 una vez alcanzado. Al leer P estando en q_0 , el no-determinismo se manifiesta como una elección: una rama permanece en q_0 (por si la condición se cumple más adelante con otra P) y otra rama pasa a q_1 declarando cumplida la condición. Basta con que una de las ramas termine en un estado de aceptación para aceptar la cadena.

4. Pruebas

Cadena	Resultado esperado
P	Pertenece
AP	Pertenece
ABP	Pertenece
PABA	Pertenece
A	No pertenece
B	No pertenece
AB	No pertenece
BAA	No pertenece

5. Código

```

6. /*
7.  * AFN - Ejercicio 2: Categoría de un producto
8.  * Acepta las cadenas que tengan por lo menos una P.
9.  */
10.
11. #include <iostream>
12. #include <string>
13. using namespace std;
14.
15. bool simularAFN(string cadena) {
16.     bool encontroP = false;
17.
18.     for (int i = 0; i < cadena.length(); i++) {
19.         if (cadena[i] == 'P') {
20.             encontroP = true;
21.         }
22.     }
23.
24.     return encontroP;
25. }
26.

```

```

27.int main() {
28.    int cantidad;
29.
30.    cout << "AFN - Categoria de producto (debe contener al menos una
    P)\n";
31.    cout << "Ingrese el numero de cadenas a probar: ";
32.    cin >> cantidad;
33.
34.    for (int i = 0; i < cantidad; i++) {
35.        string cadena;
36.        cout << "Cadena " << i + 1 << ": ";
37.        cin >> cadena;
38.
39.        bool valida = true;
40.
41.        for (int j = 0; j < cadena.length(); j++) {
42.            if (cadena[j] != 'A' && cadena[j] != 'B' && cadena[j] !=
                'P') {
43.                valida = false;
44.                break;
45.            }
46.        }
47.
48.        if (!valida) {
49.            cout << "    -> Cadena con simbolos fuera de {A,B,P}.
                Rechazada.\n";
50.        }
51.        else if (simularAFN(cadena)) {
52.            cout << "    -> ACEPTADA (pertenece a la categoria)\n";
53.        }
54.        else {
55.            cout << "    -> RECHAZADA\n";
56.        }
57.    }
58.
59.    return 0;
60.}
61.

```

C) Registro para una actividad especial

1. Alfabeto

$\Sigma = \{A, B, C\}$

2. Estados:

- q_0 : Inicial, ya leyó el primer símbolo.
- q_1 : Aceptación, ya se leyó la primer letra valida.

3. Tabla de transición

Estado	A	B	P
$\rightarrow q_0$	$\{q_1\}$	$\{q_1\}$	$\emptyset$
$* q_1$	$\{q_1\}$	$\{q_1\}$	$\{q_1\}$

desde q_0 , tanto A como B llevan al mismo estado de aceptación q_1 —aquí el no-determinismo se ve como una elección entre dos ramas válidas para el primer símbolo—, mientras que C no tiene transición definida desde q_0 y la cadena se rechaza de inmediato. Una vez en q_1 , los tres símbolos mantienen el automata en q_1 .

4. Pruebas

Cadena	Resultado esperado
A	Valido
B	Valido
ABV	Valido
BAAC	Valido
BCBA	Valido
C	No valido
CA	No valido
AB	No valido
CCAB	No valido

5. Código

```
6. /*
7.  * AFN - Ejercicio 3: Actividad especial
8.  * La cadena debe comenzar con A o con B.
9.  */
10.
11. #include <iostream>
12. #include <string>
13. using namespace std;
14.
```

```
15. bool simularAFN(string cadena) {
16.     if (cadena.length() == 0)
17.         return false;
18.
19.     if (cadena[0] != 'A' && cadena[0] != 'B')
20.         return false;
21.
22.     for (int i = 0; i < cadena.length(); i++) {
23.         if (cadena[i] != 'A' && cadena[i] != 'B' && cadena[i] != 'C')
24.             return false;
25.     }
26.
27.     return true;
28. }
29.
30. int main() {
31.     int cantidad;
32.
33.     cout << "AFN - Registro actividad especial (debe iniciar con A o
    B)\n";
34.     cout << "Ingrese el numero de cadenas a probar: ";
35.     cin >> cantidad;
36.
37.     for (int i = 0; i < cantidad; i++) {
38.         string cadena;
39.         cout << "Cadena " << i + 1 << ": ";
40.         cin >> cadena;
41.
42.         bool valida = !cadena.empty();
43.
44.         for (int j = 0; j < cadena.length(); j++) {
45.             if (cadena[j] != 'A' && cadena[j] != 'B' && cadena[j] !=
                'C') {
46.                 valida = false;
47.                 break;
48.             }
49.         }
50.
51.         if (!valida) {
52.             cout << "    -> Cadena vacia o con simbolos fuera de
                {A,B,C}. Rechazada.\n";
53.         }
54.         else if (simularAFN(cadena)) {
55.             cout << "    -> ACEPTADA (comienza con A o con B)\n";
56.         }
```

```
57.         else {
58.             cout << "    -> RECHAZADA\n";
59.         }
60.     }
61.
62.     return 0;
63. }
64.
```